

```

const express = require('express');
const cors = require('cors');
const fetch = require('node-fetch');
const tough = require('tough-cookie');
const fetchCookie = require('fetch-cookie');

const app = express();
app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 3000;

// Health check
app.get('/', (req, res) => {
  res.json({ status: 'Atleet Garmin Proxy actief ✓' });
});

// Main endpoint: login + fetch all data
app.post('/garmin/sync', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ error: 'E-mail en wachtwoord vereist' });
  }

  try {
    // Create cookie jar for session management
    const cookieJar = new tough.CookieJar();
    const fetchWithCookies = fetchCookie(fetch, cookieJar);

    console.log('🔒 Inloggen voor:', email);

    // Step 1: Get SSO page + CSRF token
    const ssoParams = new URLSearchParams({
      service: 'https://connect.garmin.com/modern/',
      webhost: 'olaxpw-conctmodern005',
      source: 'https://connect.garmin.com/signin',
      redirectAfterAccountLoginUrl: 'https://connect.garmin.com/modern/',
      redirectAfterAccountCreationUrl: 'https://connect.garmin.com/modern/',
      gauthHost: 'https://sso.garmin.com/sso',
      locale: 'en_US',
      id: 'gauth-widget',
      clientId: 'GarminConnect',
      rememberMeShown: 'true',
      rememberMeChecked: 'false',
    });
  }
});

```

```

    createAccountShown: 'true',
    openCreateAccount: 'false',
    consumeServiceTicket: 'false',
    initialFocus: 'true',
    embedWidget: 'false',
    generateExtraServiceTicket: 'true',
    generateTwoExtraServiceTickets: 'false',
    generateNoServiceTicket: 'false',
    globalOptInShown: 'true',
    globalOptInChecked: 'false',
    mobile: 'false',
    connectLegalTerms: 'true',
    showPassword: 'true',
    useCustomHeader: 'false',
  });

const ssoUrl = `https://sso.garmin.com/sso/signin?${ssoParams}`;

const ssoRes = await fetchWithCookies(ssoUrl, {
  headers: {
    'User-Agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) App
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.
    'Accept-Language': 'nl-NL,nl;q=0.9,en;q=0.8',
  }
});

const ssoHtml = await ssoRes.text();
const csrfMatch = ssoHtml.match(/name="_csrf"\s+value="([^"]+)"/);
if (!csrfMatch) {
  throw new Error('Kon CSRF token niet vinden - Garmin SSO pagina onbereikbaar');
}
const csrf = csrfMatch[1];
console.log('✓ CSRF token:', csrf.substring(0, 8) + '...');

// Step 2: POST login
const formData = new URLSearchParams();
formData.append('username', email);
formData.append('password', password);
formData.append('embed', 'false');
formData.append('_csrf', csrf);

const loginRes = await fetchWithCookies(ssoUrl, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'User-Agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) App
    'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.

```

```

    'Origin': 'https://sso.garmin.com',
    'Referer': ssoUrl,
    'NK': 'NT',
  },
  body: formData.toString(),
  redirect: 'follow',
});

const loginHtml = await loginRes.text();

if (loginHtml.includes('Invalid') || loginHtml.includes('incorrect') || login
    throw new Error('Onjuist e-mailadres of wachtwoord');
}

const ticketMatch = loginHtml.match(/ticket=([A-Za-z0-9_-]+)/);
if (!ticketMatch) {
  throw new Error('Login mislukt – geen ticket ontvangen. Controleer je gegev
}
const ticket = ticketMatch[1];
console.log('✓ Ticket ontvangen');

// Step 3: Exchange ticket for session cookies
const modernUrl = `https://connect.garmin.com/modern/?ticket=${ticket}`;
await fetchWithCookies(modernUrl, {
  headers: {
    'User-Agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) App
  },
  redirect: 'follow',
});
console.log('✓ Sessie aangemaakt');

// Helper for authenticated GC requests
const gcFetch = async (path) => {
  const url = 'https://connect.garmin.com' + path;
  const r = await fetchWithCookies(url, {
    headers: {
      'User-Agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) A
      'NK': 'NT',
      'X-Requested-With': 'XMLHttpRequest',
      'Accept': 'application/json, text/javascript, */*; q=0.01',
      'Referer': 'https://connect.garmin.com/modern/',
    }
  });
  if (!r.ok) throw new Error(`${r.status} voor ${path}`);
  return r.json();
};

```

```

const today = new Date().toISOString().split('T')[0];
const result = { date: today, synced: [] };

// Get display name for sleep endpoint
let displayName = email.split('@')[0];
try {
  const profile = await gcFetch('/proxy/userprofile-service/socialProfile');
  displayName = profile.displayName || displayName;
  result.profile = { displayName, fullName: profile.fullName };
  console.log('✓ Profiel:', displayName);
} catch(e) { console.log('! Profiel:', e.message); }

// Daily summary
try {
  const daily = await gcFetch(`/proxy/usersummary-service/usersummary/daily/$
result.daily = {
  stappen: daily.totalSteps || 0,
  stress: daily.averageStressLevel || 0,
  kcalVerbrand: daily.activeKilocalories || 0,
  bmr: daily.bmrKilocalories || 0,
  intensiteitsMinuten: daily.moderateIntensityMinutes || 0,
};
result.synced.push(`👣 ${((daily.totalSteps||0).toLocaleString())} stappen`);
console.log('✓ Dagdata');
} catch(e) { console.log('! Dagdata:', e.message); result.daily = null; }

// Sleep
try {
  const sleep = await gcFetch(`/proxy/wellness-service/wellness/dailySleepDat
if (sleep && sleep.dailySleepDTO) {
  const s = sleep.dailySleepDTO;
  result.sleep = {
    duur: Math.round((s.sleepTimeSeconds || 0) / 360) / 10,
    diepe: Math.round((s.deepSleepSeconds || 0) / 60),
    lichte: Math.round((s.lightSleepSeconds || 0) / 60),
    rem: Math.round((s.remSleepSeconds || 0) / 60),
    wakker: Math.round((s.awakeSleepSeconds || 0) / 60),
    score: s.sleepScores?.overall?.value || 0,
    startTijd: s.sleepStartTimestampLocal,
    eindTijd: s.sleepEndTimestampLocal,
  };
  result.synced.push(`😴 ${result.sleep.duur}u slaap`);
  console.log('✓ Slaap:', result.sleep.duur + 'u');
}
} catch(e) { console.log('! Slaap:', e.message); result.sleep = null; }

// Heart rate

```

```

try {
  const hr = await gcFetch(`/proxy/wellness-service/wellness/dailyHeartRate/$
  result.hartslag = {
    rhr: hr.restingHeartRate || 0,
    max: hr.maxHeartRate || 0,
    min: hr.minHeartRate || 0,
  };
  result.synced.push(`💖 ${hr.restingHeartRate || '-'} bpm RHR`);
  console.log('✓ Hartslag RHR:', hr.restingHeartRate);
} catch(e) { console.log('⚠️ Hartslag:', e.message); result.hartslag = null;

// HRV
try {
  const hrv = await gcFetch(`/proxy/hrv-service/hrv/${today}`);
  if (hrv && hrv.hrvSummary) {
    result.hrv = {
      nachtelijk: hrv.hrvSummary.lastNight || 0,
      status: hrv.hrvSummary.status || '',
      weeklyAvg: hrv.hrvSummary.weeklyAvg || 0,
      feedback: hrv.hrvSummary.feedback || '',
    };
    result.synced.push(`📊 HRV ${result.hrv.nachtelijk}ms`);
    console.log('✓ HRV:', result.hrv.nachtelijk + 'ms');
  }
} catch(e) { console.log('⚠️ HRV:', e.message); result.hrv = null; }

// VO2max
try {
  const vo2 = await gcFetch(`/proxy/metrics-service/metrics/maxmet/weekly/${t
  if (vo2?.allMetrics?.metricsMap) {
    const map = vo2.allMetrics.metricsMap;
    const val = (map.VO2_MAX_RUNNING || map.VO2_MAX_CYCLING || [])[0]?.value;
    if (val) { result.vo2max = val; result.synced.push(`🫁 VO2max ${val}`); }
    console.log('✓ VO2max:', val);
  }
} catch(e) { console.log('⚠️ VO2max:', e.message); result.vo2max = null; }

// Activities (last 30)
try {
  const acts = await gcFetch('/proxy/activitylist-service/activities/search/a
  if (Array.isArray(acts)) {
    const sportMap = {
      'running': '🏃 Hardlopen', 'trail_running': '🏃 Hardlopen',
      'cycling': '🚲 Fietsen', 'indoor_cycling': '🚲 Fietsen', 'virtual_ride'
      'strength_training': '🏋️ Gym', 'fitness_equipment': '🏋️ Gym',
      'hiit': '⚡ HIIT/Hyrox', 'cardio_training': '⚡ HIIT/Hyrox',
      'walking': '🚶 Wandelen', 'swimming': '🏊 Zwemmen',

```

```

    };
    result.activiteiten = acts.map(a => ({
      garmin_id: a.activityId,
      naam: a.activityName || '',
      sport: sportMap[a.activityType?.typeKey?.toLowerCase()] || a.activityTy
      datum: a.startTimeLocal || a.beginTimestamp,
      duur: Math.round((a.duration || 0) / 60),
      afstand: parseFloat(((a.distance || 0) / 1000).toFixed(2)),
      hartslag: a.averageHR || 0,
      maxHartslag: a.maxHR || 0,
      kcalVerbrand: a.calories || 0,
      elevatie: a.elevationGain || 0,
      vo2max: a.vO2MaxValue || 0,
    }));
    result.synced.push(`🏃 ${acts.length} activiteiten`);
    console.log('✓ Activiteiten:', acts.length);
  }
} catch(e) { console.log('⚠ Activiteiten:', e.message); result.activiteiten

console.log('✅ Sync klaar voor', email);
res.json(result);

} catch (err) {
  console.error('❌ Sync fout:', err.message);
  res.status(500).json({ error: err.message });
}
});

app.listen(PORT, () => {
  console.log(`🚀 Garmin proxy draait op poort ${PORT}`);
});

```